MICRO 2026

TensorMLD: A Tensorized and Compressed qLDPC Maximum-Likelihood Decoder

Liqiang Lu*, Debin Xiang*, Kaiwen Zhou*, Boyu Jin*, Chenning Tao*, Wuwei Tian*,
Anbang Wu§, Fangxin Liu§, Jingwen Leng§, Minyi Guo§, Jianwei Yin*
*Zhejiang University; §Shanghai Jiao Tong University

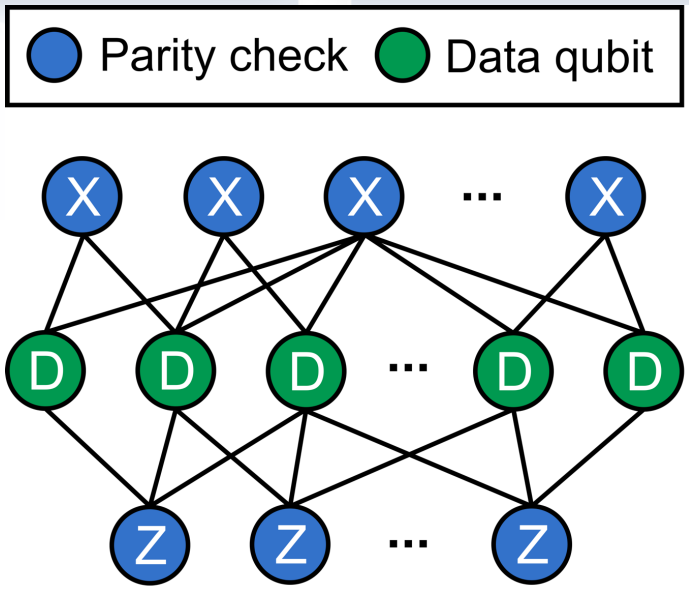
College of Computer Science and Technology
Zhejiang University (ZJU)

Outline of Presentation

1. **Background and motivation**
2. Tensorization and offline compression
3. Online decoding and hardware architecture
4. Evaluation and conclusions

Background: qLDPC Decoding

Encode many physical qubit into logical operotors → **Measure stabilizers → syndrome s**

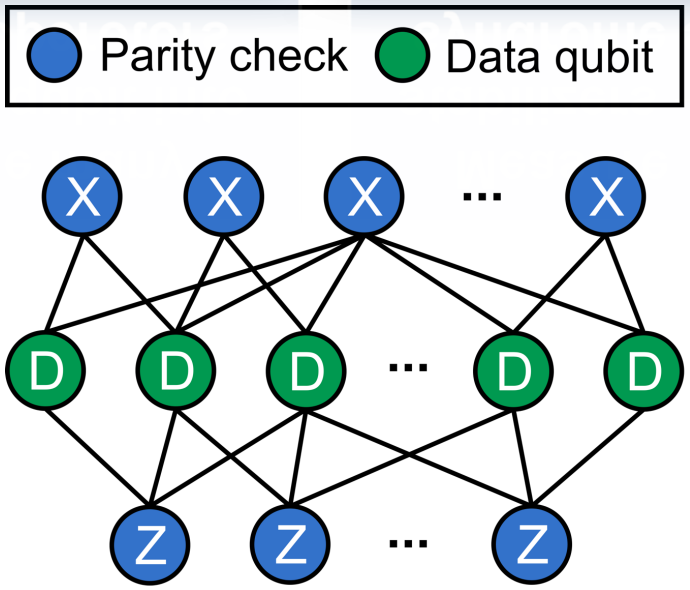


Tanner graph of general qLDPC codes

Data qubits has errors and we measure the parity check qubits to get information of these error locations and errot type

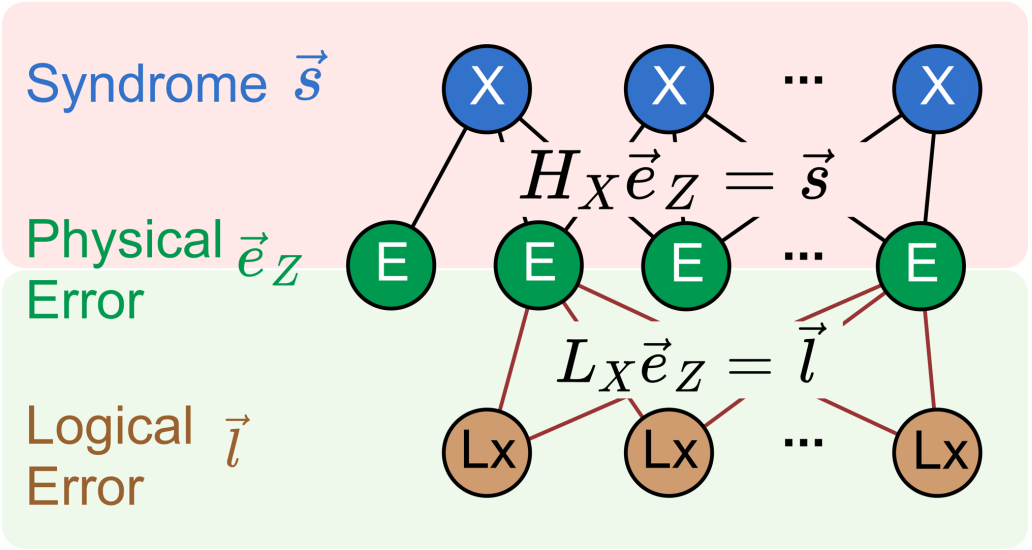
Background: qLDPC Decoding

Encode many physical qubit into logical operotors → Measure stabilizers → syndrome s → Infer the most likely logical error → Apply correction operator to recover logical qubit



Tanner graph of general qLDPC codes

Decoding Pauli-Z errors

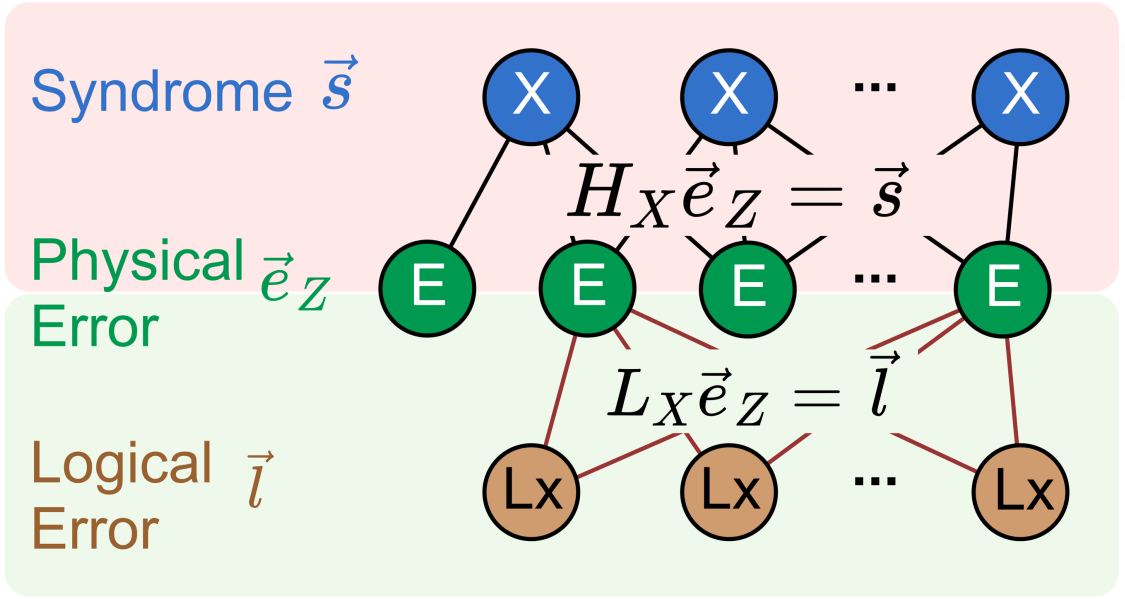


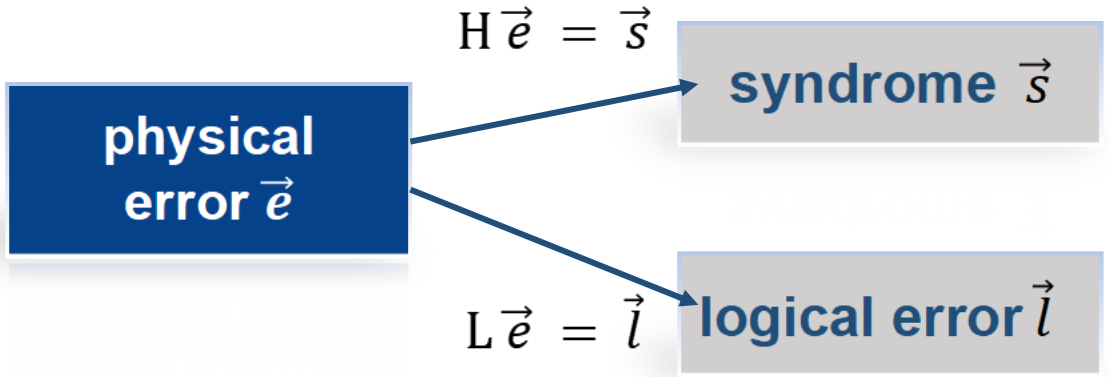
Relationship between physical errors, logical errors, and syndrome

Non-local checks make qLDPC decoding NP-hard. The objective is to find most likely logical errors (Maximum-Likelihood Decoder,MLD)

Decoding Relation and the MLD Objective

Decoding Pauli-Z errors





$$P(\vec{l} \mid \vec{s}) = \Sigma\, P(\vec{e})$$

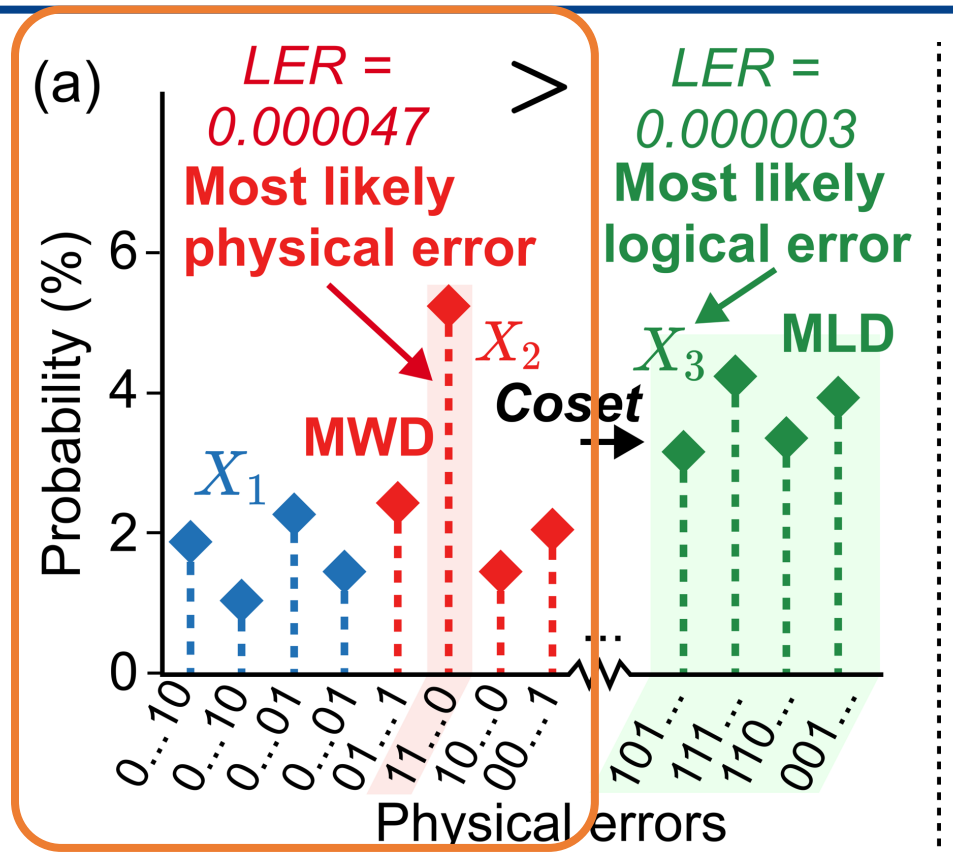
over every e with $H\vec{e} = \vec{s}$ and $L\vec{e} = \vec{l}$

Most likely logical operator:

$$\vec{l}^{*} = \text{argmax}\ P(\vec{l} \mid \vec{s})$$

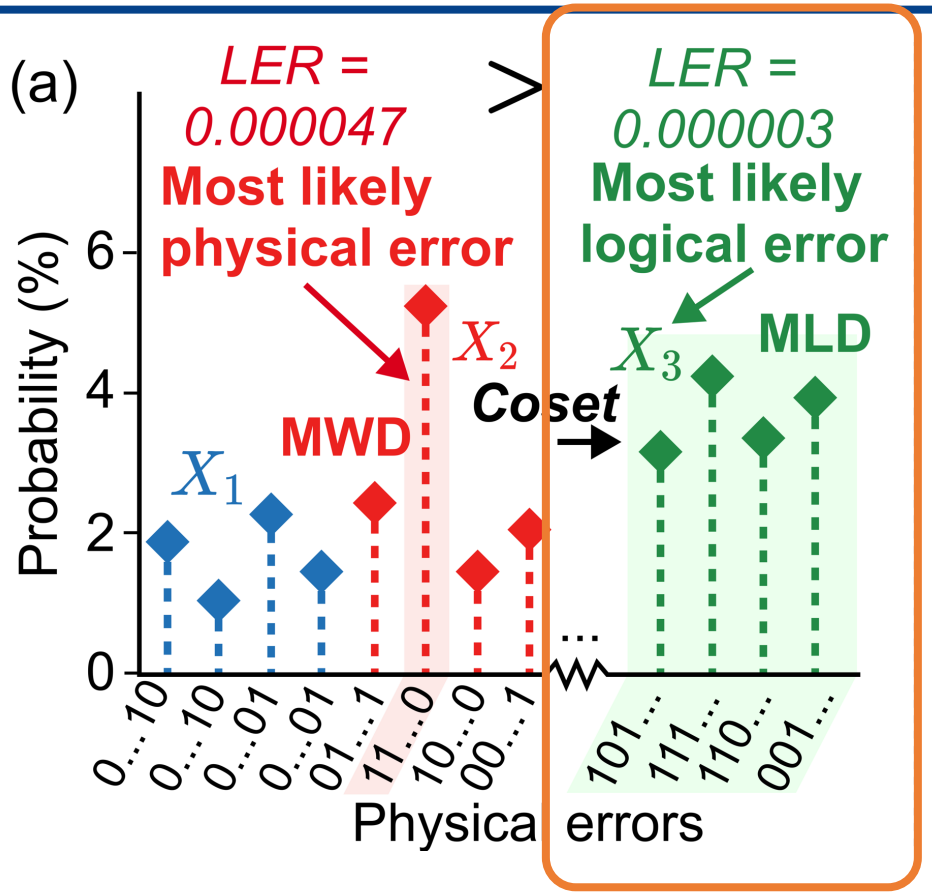
MLD counts the entire physical errors producing syndromes and belongs to same logical erros, exponentially many terms with additional two constriants equations

Motivation: MWD Is Not MLD



MWD: picks X2, the single most probable physical error

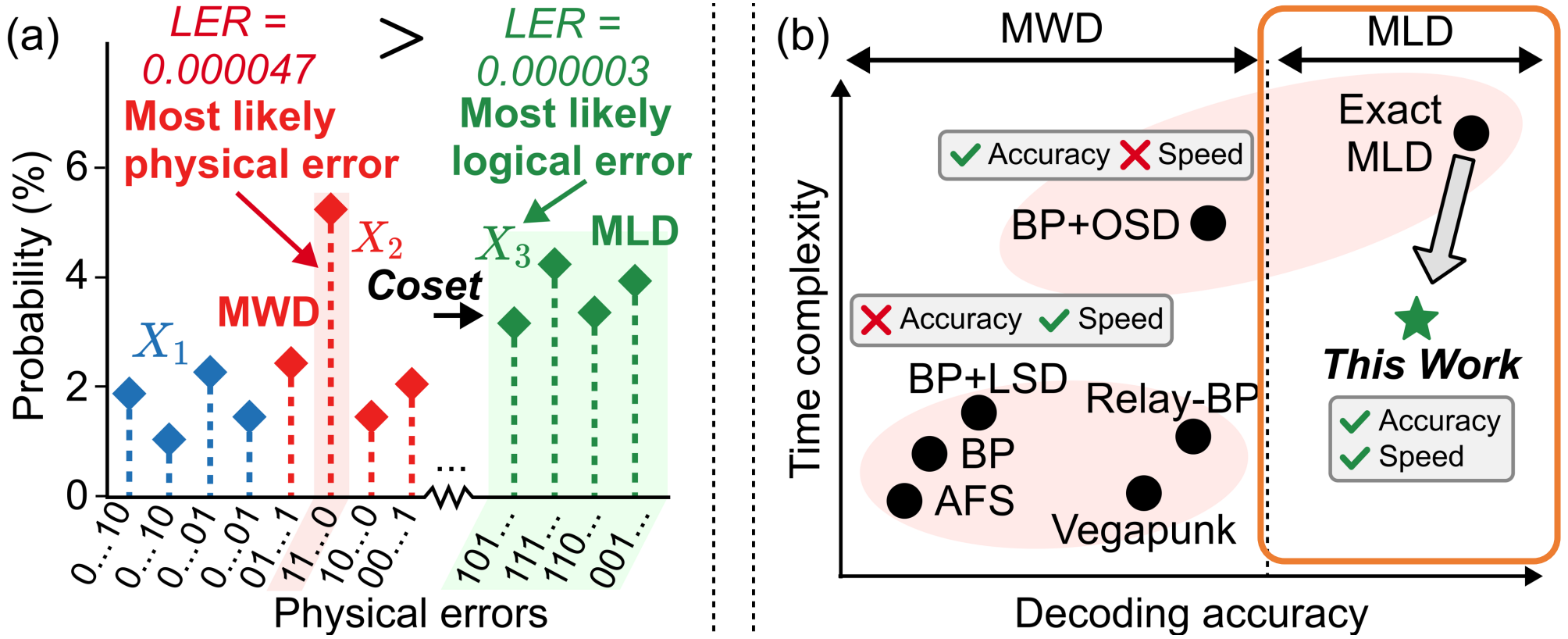
Motivation: MWD Is Not MLD



MWD: picks X3, the single most probable physical error

Minimal weight decoder tends to find the most probable physical error

Motivation: MWD Is Not MLD



MWD: picks X2, the single most probable physical error

MLD: sums each coset — optimal but exponential

Most likely physical error ≠ most likely logical error

Challenges: Accuracy Gap and Computation Wall

ZHEJIANG UNIVERSITY

Accuracy gap

18.4%

BP+OSD recovers the true logical error on the 72-qubit BB code only 18.4% of the time

Computation wall

$O(2^n)$

High-rank tensors and logical-state enumeration grow exponentially with code size

MWD accelerators (early termination, pruning) corrupt MLD: LER $10^{-6} \rightarrow 10^{-4}$

Goal: MLD-level accuracy with polynomial online complexity

Application of MLD decoder

1. Lower physical-qubit overhead.
2. Less demanding hardware fidelity requirements.
3. longer computations
4. A benchmark for evaluating codes

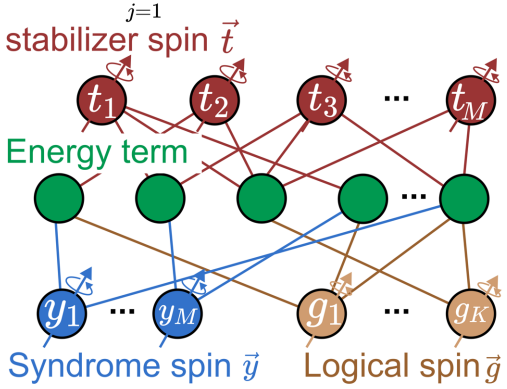
TensorMLD Overview

1 Tensorize MLD

spin-glass model

Spin glass Hamiltonian

$$H = -\sum_{j=1}^{n} \omega_j \prod t_k \prod y_k \prod g_k$$



TensorMLD Overview

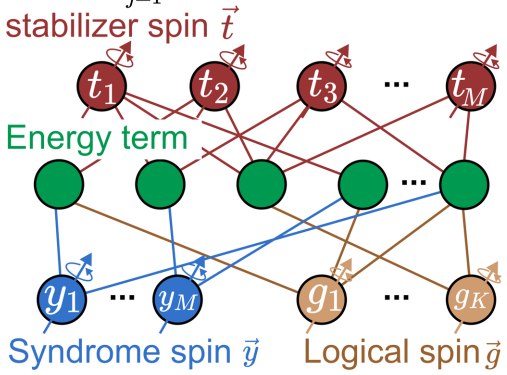
1 Tensorize MLD

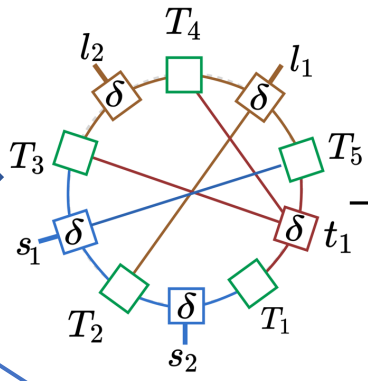
spin-glass model

tensor network

Spin glass Hamiltonian

$$H = -\sum_{j=1}^{n} \omega_j \prod t_k \prod y_k \prod g_k$$





But original tensor network has high-demensional tensors, resulting large storage and computation cost

TensorMLD Overview

1 Tensorize MLD → **2 Exact decompostion**

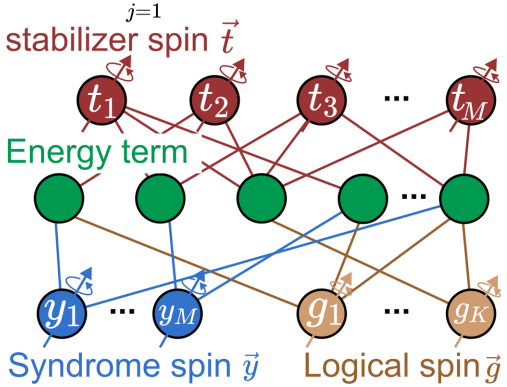
spin-glass model

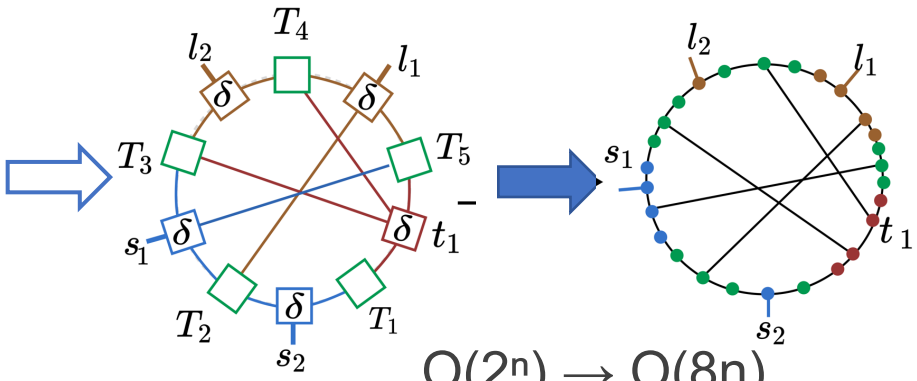
tensor network

matrix product states

Spin glass Hamiltonian

$$H = -\sum_{j=1}^{n} \omega_j \prod t_k \prod y_k \prod g_k$$





$O(2^n) \longrightarrow O(8n)$

After decomposition, the good thing is that each tensor get smaller. However, the bad thing is that we get more tensors in matrix product states

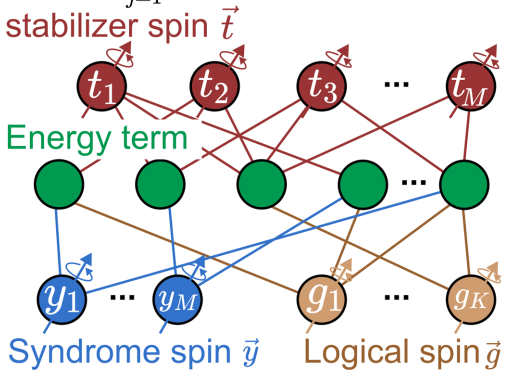
TensorMLD Overview

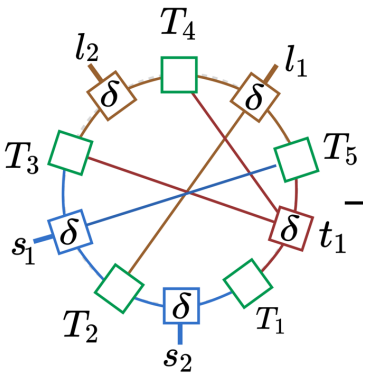
1 Tensorize MLD → **2 Exact decompostion** — **3 Offline compression**

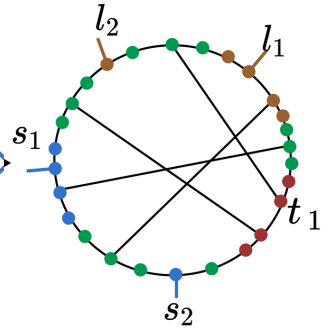
spin-glass model | tensor network | matrix product states | matrix product states chain

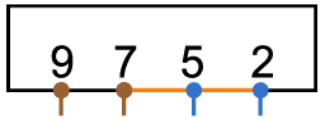
Spin glass Hamiltonian

$$H = -\sum_{j=1}^{n} \omega_j \prod t_k \prod y_k \prod g_k$$









1D-chain matrix product state

Eventally,we get a much simpler tensor network, but how to use it for decoding is a problem

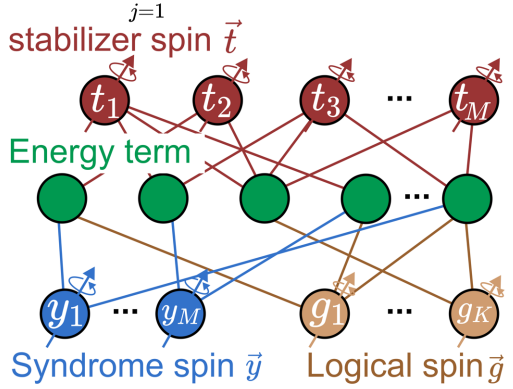
TensorMLD Overview

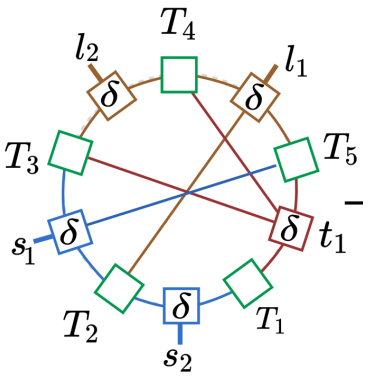
1 Tensorize MLD → **2 Exact decompostion** → **3 Offline compression**

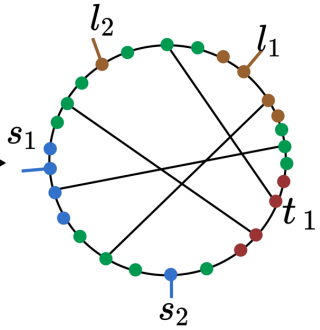
spin-glass model | tensor network | matrix product states | matrix product states chain

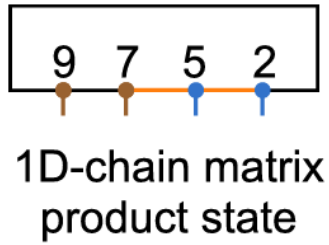
Spin glass Hamiltonian

$$H = -\sum_{j=1}^{n} \omega_j \prod t_k \prod y_k \prod g_k$$









Offline computation, the resulting compressed 1D-chain MPS can be reused for same qldpc code and same noise model

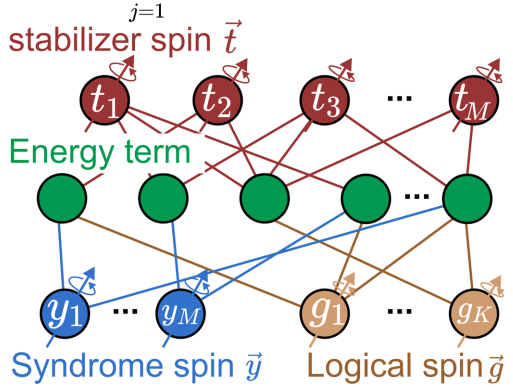
TensorMLD Overview

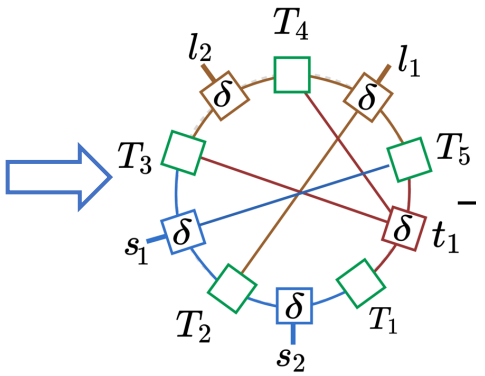
1 Tensorize MLD → **2 Exact decompostion** → **3 Offline compression** → **4 Online decoding**

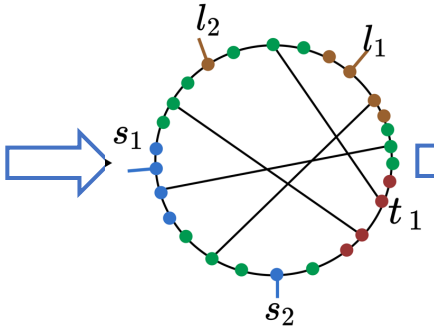
spin-glass model | tensor network | matrix product states | matrix product states chain

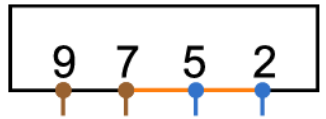
Spin glass Hamiltonian

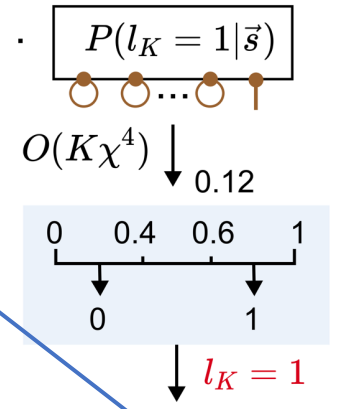
$$H = -\sum_{j=1}^{n} \omega_j \prod t_k \prod y_k \prod g_k$$







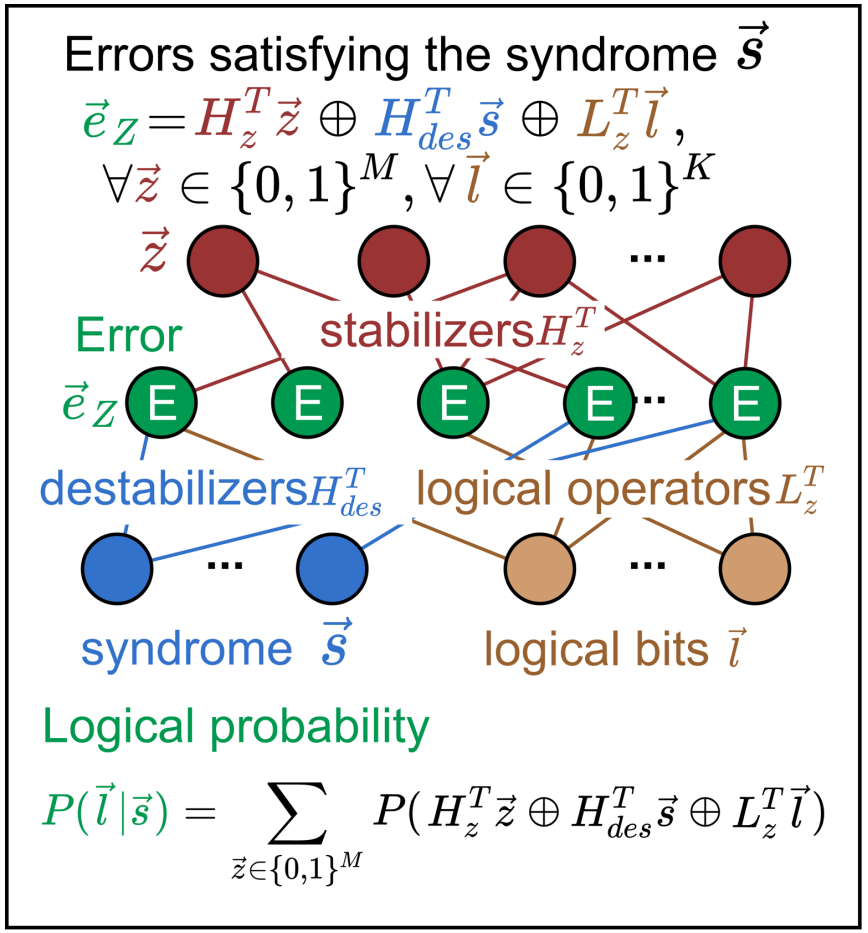




We designed a online decoding algorithm without enumerating the logical indices

A reusable 1D-chain MPS connects algorithm and hardware

Step 1: Unconstrained formulation of MLD



(a) qLDPC decoding

Bits to spins

$(-1)^{z_k} = t_k$

$(-1)^{s_k} = y_k$

$(-1)^{l_k} = g_k$

binary	spin
0	1
1	-1

XOR to multiplication

$b_i \oplus b_j \rightleftharpoons s_i s_j$

P(l | s) = Σ P(e)
over every e with H e = s and L e = l

Two constraints in original formulation

↓

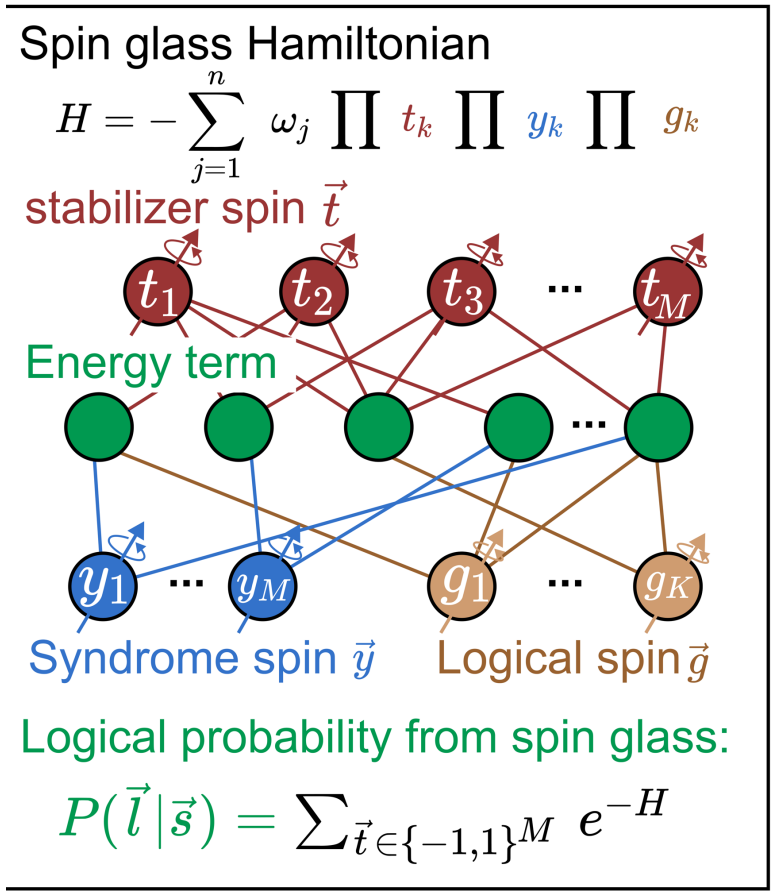
e = stabilizer part + shift(s) + logical part

↓

P(l | s) = Σ P(H_z Z+H_{des} s + L_z l)
now over ALL z — no constraints

An unconstrained sum — ready for spin-glass tensorization

Step 1: Tensorize Maximum-Likelihood Decoding

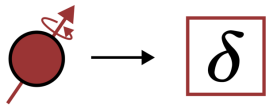


(b) Spin glass model

Energy term to energy tensor

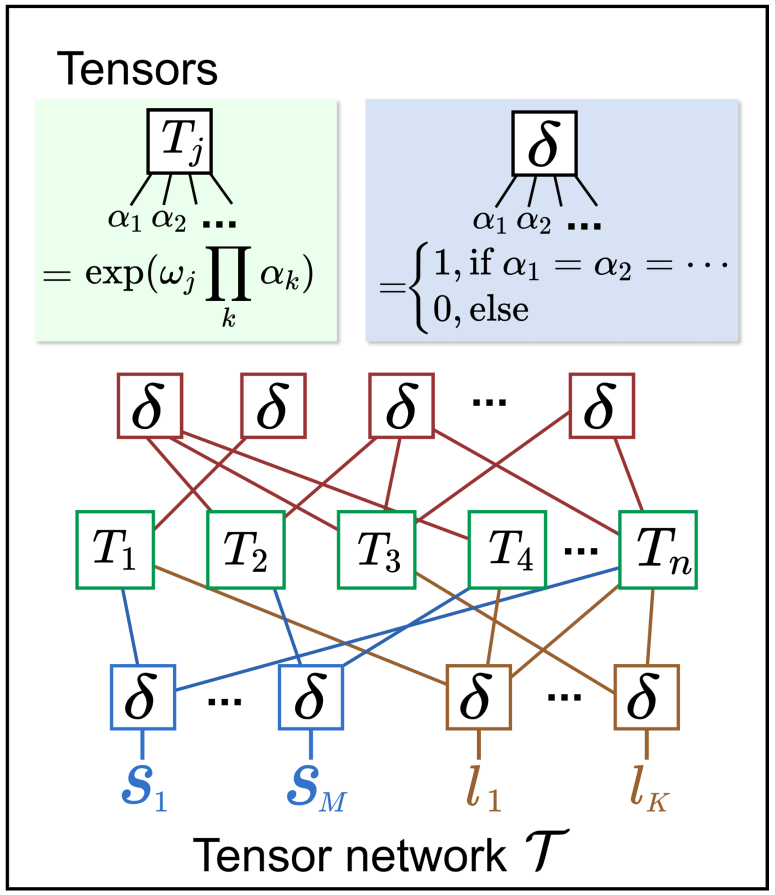


Spin variable to kronecker tensor



Summation to tensor network contraction

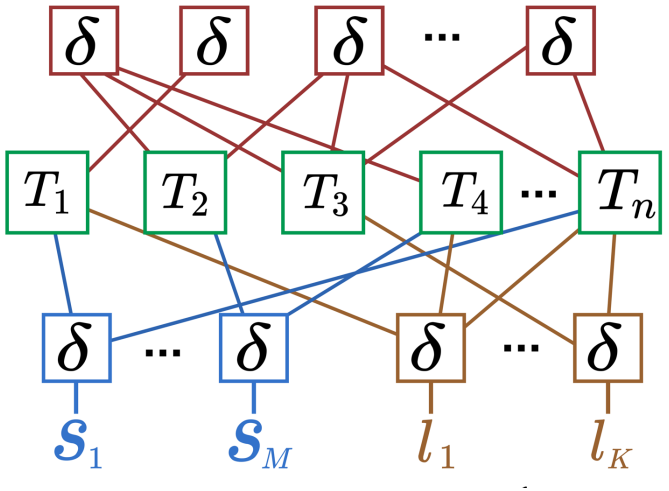
$\sum_{\vec{t}\in\{-1,1\}^M} \rightleftharpoons \text{contract}$

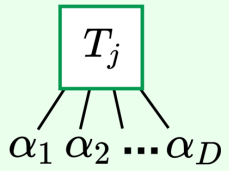


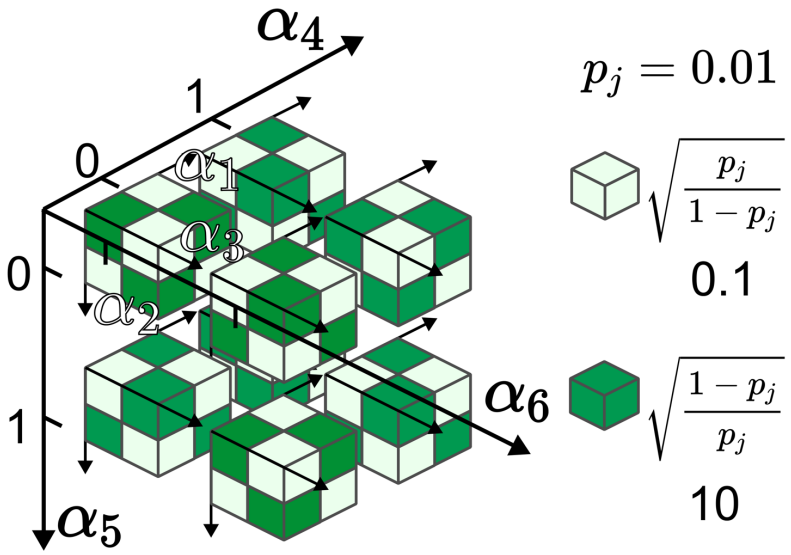
(c) Tensor network

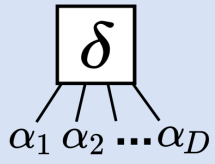
MLD objective is unchanged. The free energy is computed by tensor-network contraction

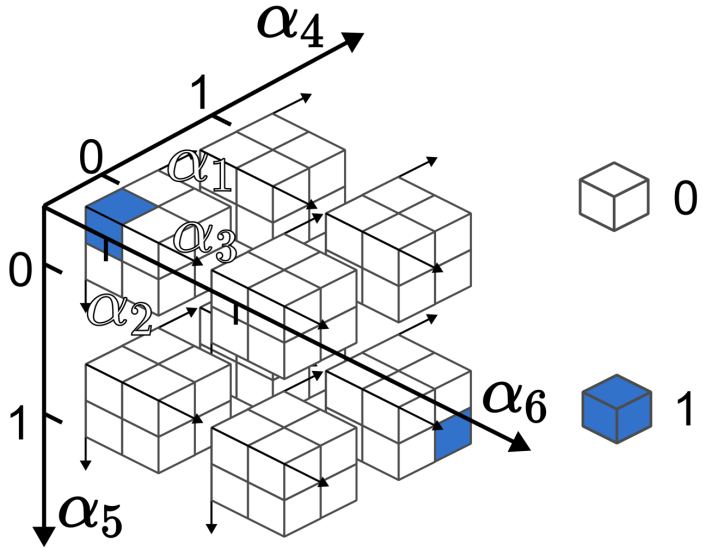
The Two Tensors, In Numbers



$T_j[\alpha_1][\alpha_2]\cdots[\alpha_6]$ 



$\delta[\alpha_1][\alpha_2]\cdots[\alpha_6]$ 

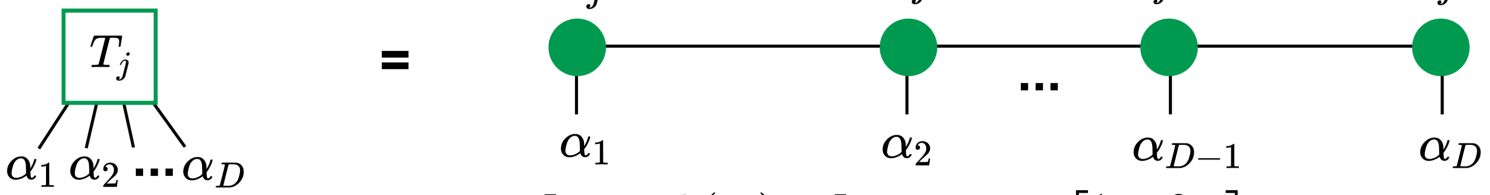


T-j: two values (≈0.1 and 10 at p = 0.01) δ: exactly two 1s

The two types of tensor has special structure, which inspire us to exactly decompose them.

Step 2: Exact Matrix-Product-State Decomposition

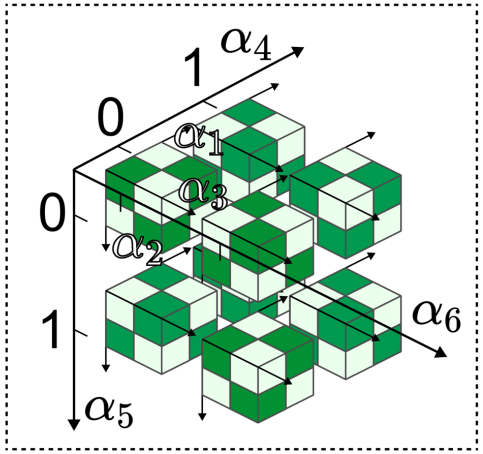
$$T_j[\alpha_1][\alpha_2]\cdots[\alpha_D]$$



$$\cosh(w_j)+\sinh(w_j)\prod_k(-1)^{\alpha_k}\quad T_j^1[\alpha_1]=\begin{bmatrix}\cosh(\omega_j)\\ \sinh(\omega_j)\cdot(-1)^{\alpha_1}\end{bmatrix}\quad T_j^k[\alpha_k]=\begin{bmatrix}1 & 0\\ 0 & (-1)^{\alpha_k}\end{bmatrix}\quad T_j^D[\alpha_D]=\begin{bmatrix}1\\ (-1)^{\alpha_D}\end{bmatrix}$$

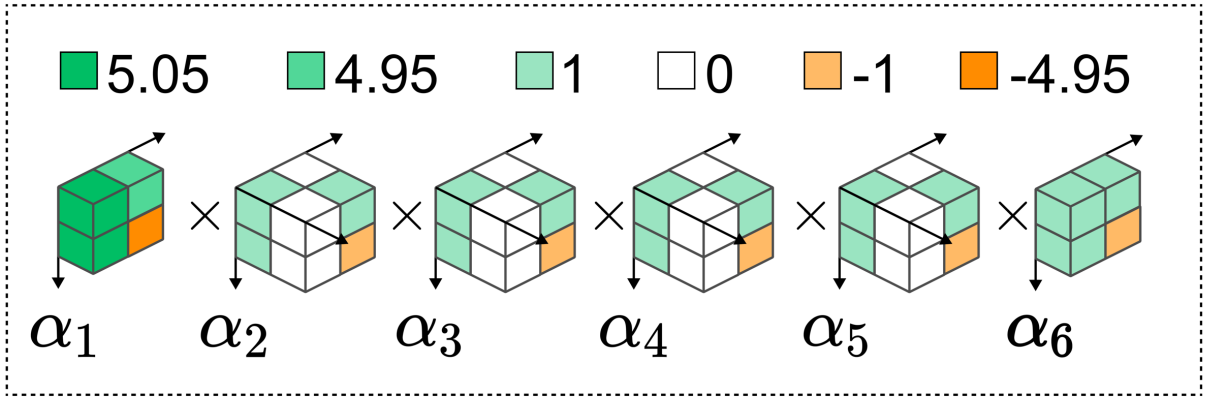
(a) Exact matrix product state decomposition of energy tensor

64 elements $O(2^n)$



=

40 elements $O(8n)$



(b) Example of energy tensor decomposition

Structured: binary, parity, diagonal

Exact MPS

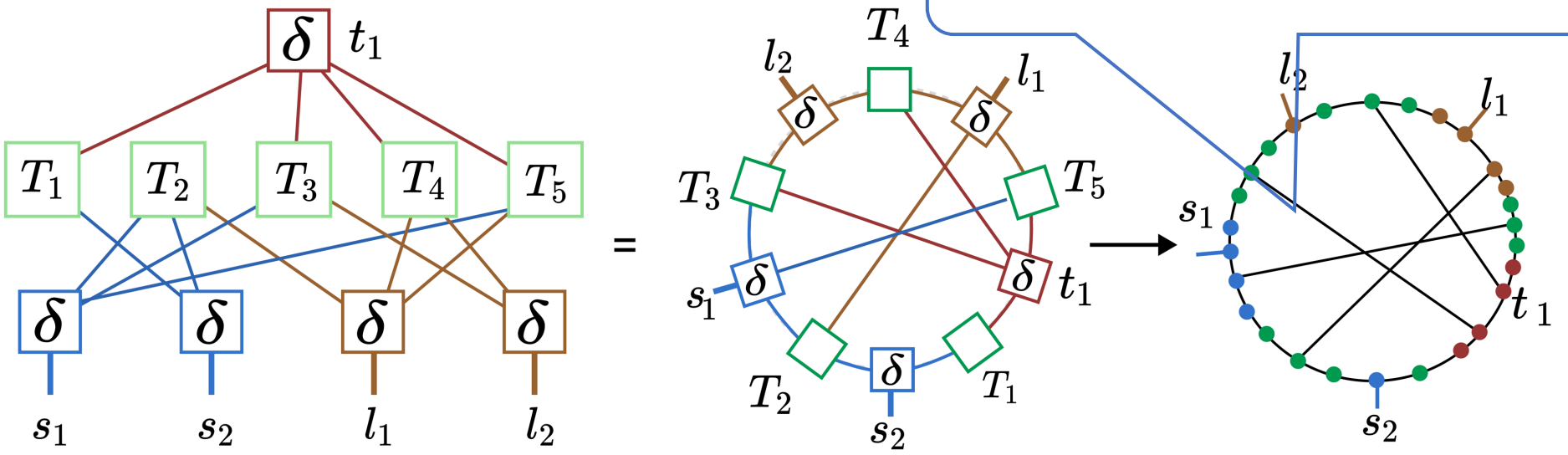
O(2ⁿ) → O(8n)

Rank 6: 64 → 40

TB 254 code: 3.9 GB → 8.7 KB

Why a 1D Chain? Circle Layout

If we remove the chords inside the circle, the network become one chain



(c) Example tensor network (d) Circle layout (e) Matrix product state

≤4 neighbors on the circle (d) vs ≤3 in the chain (e). Internal chords are what offline contraction removes

Step 3: Offline Contraction to a 1D Chain

Contract cheapest edge

↓

Swap to align duplicate edges

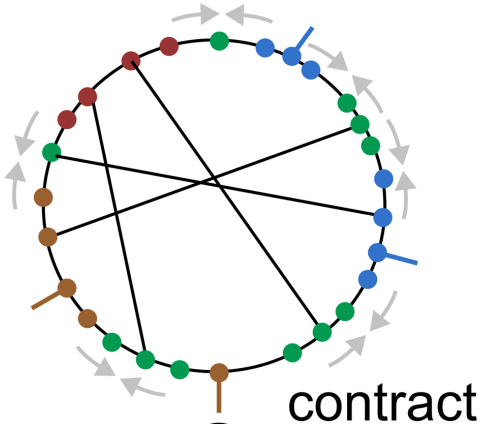
↓

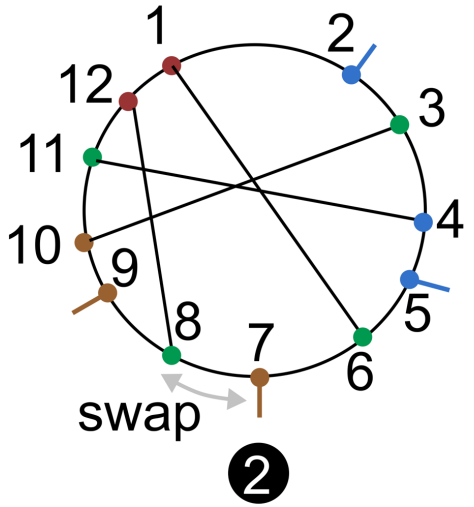
Merge aligned edges (D grows)

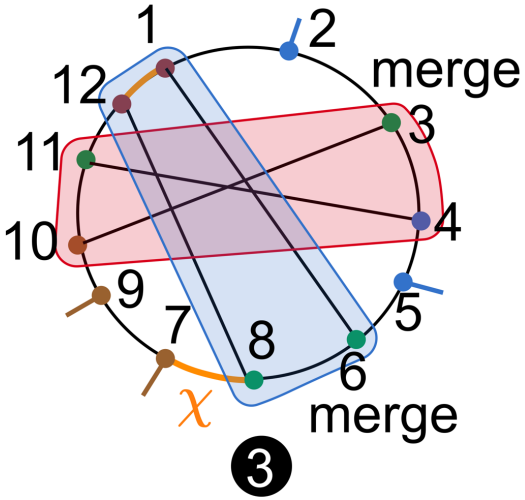
repeat

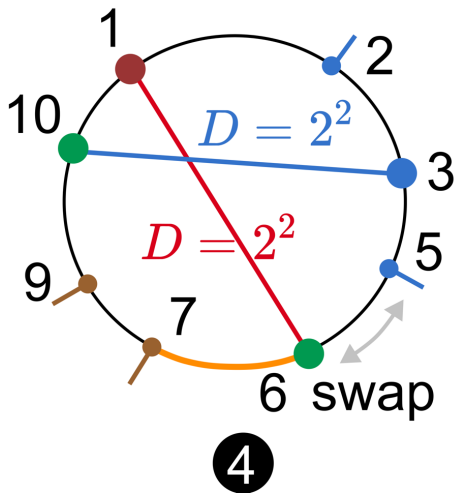
Greedy order → fewest lossy swaps; bonds capped at χ

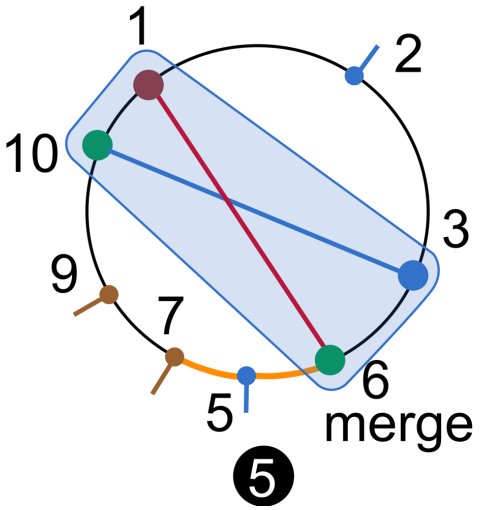
1D-chain MPS: syndrome + logical

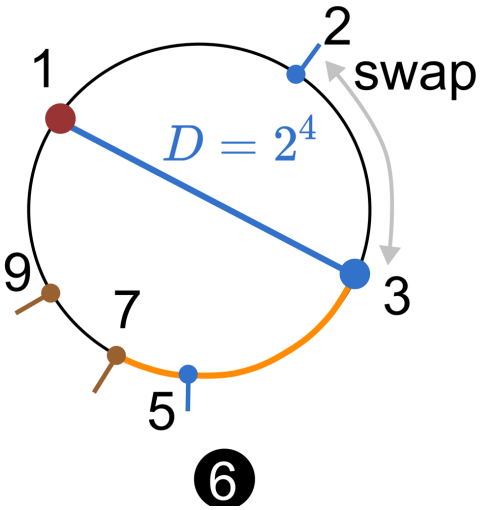






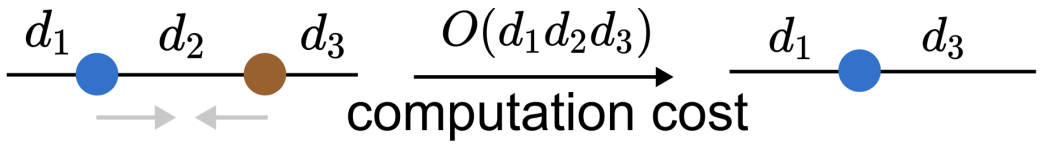






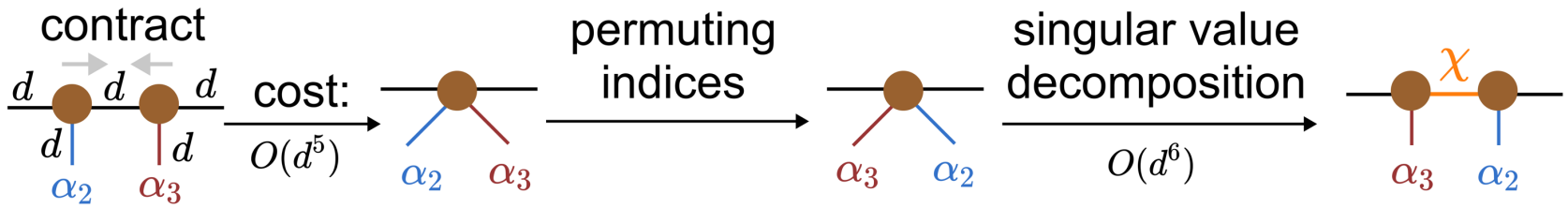
Contract, Swap, Merge — and Their Costs

Contract: $O(d_1d_2d_3)$ — exact

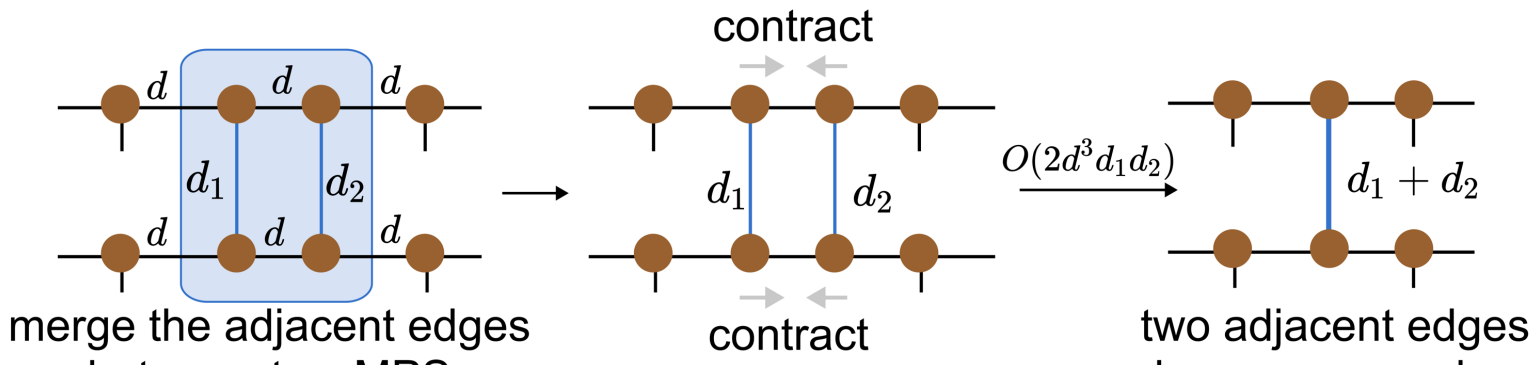


Swap:
SVD, $O(d^6)$
the only lossy step

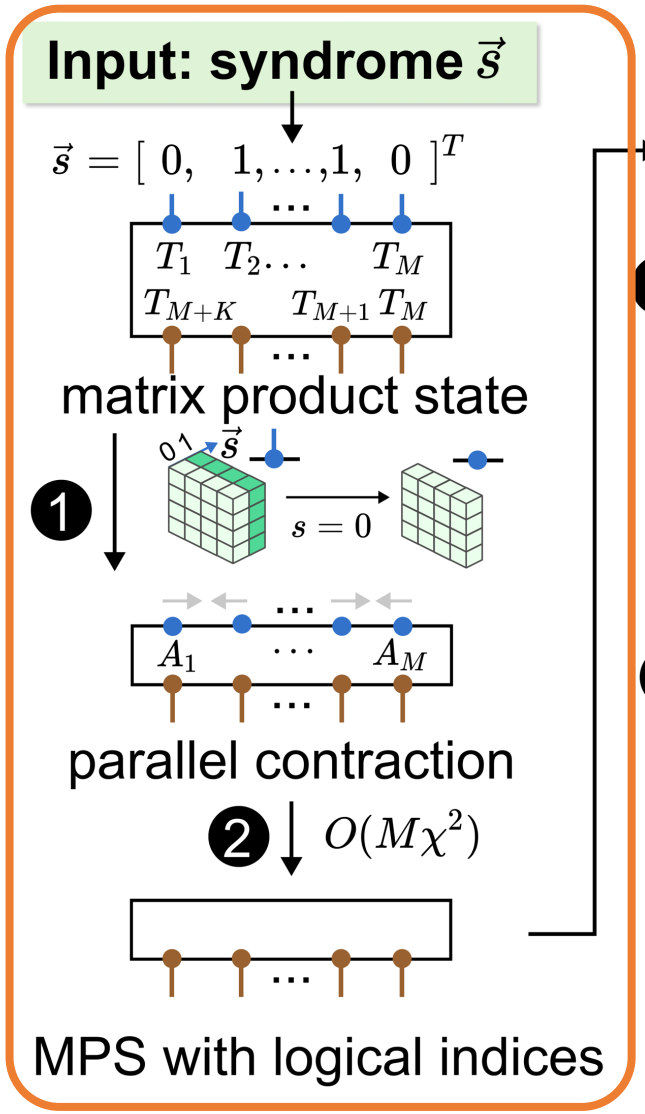
Swap between two common tensors has cost and accuracy loss

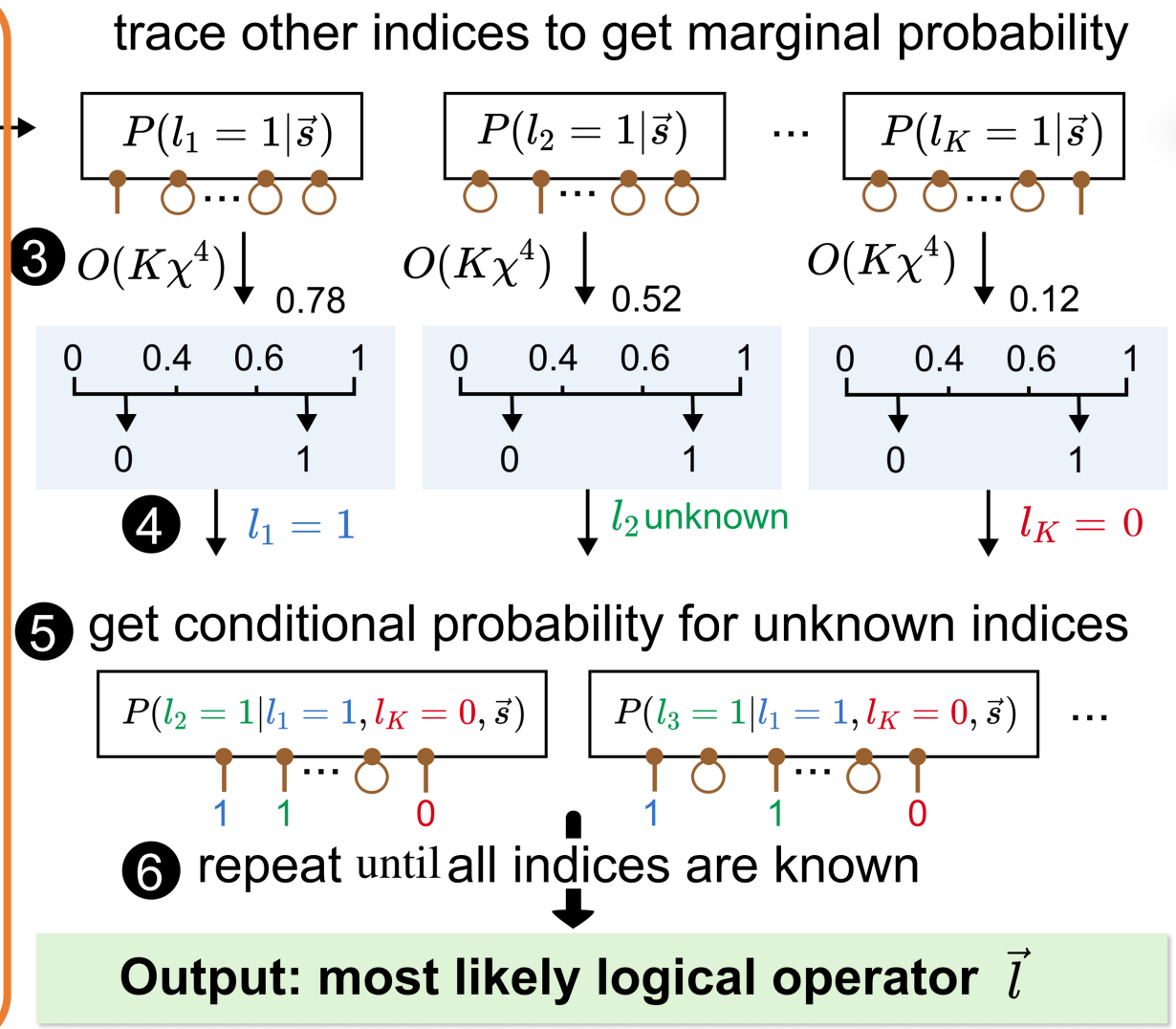


Merge:
two contracts,
exact, D grows



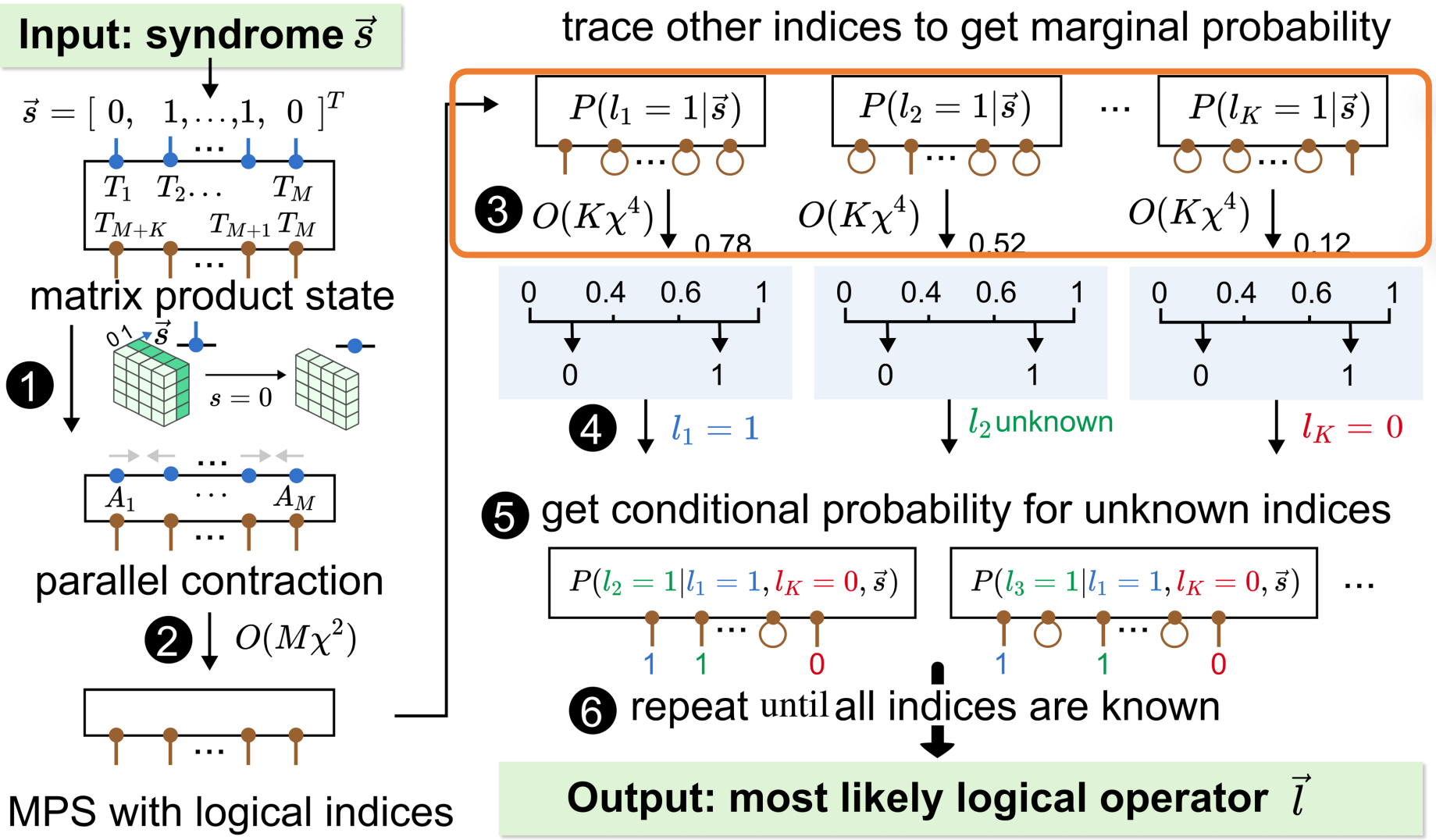
Step 4: Online Parallel Marginal Decoding



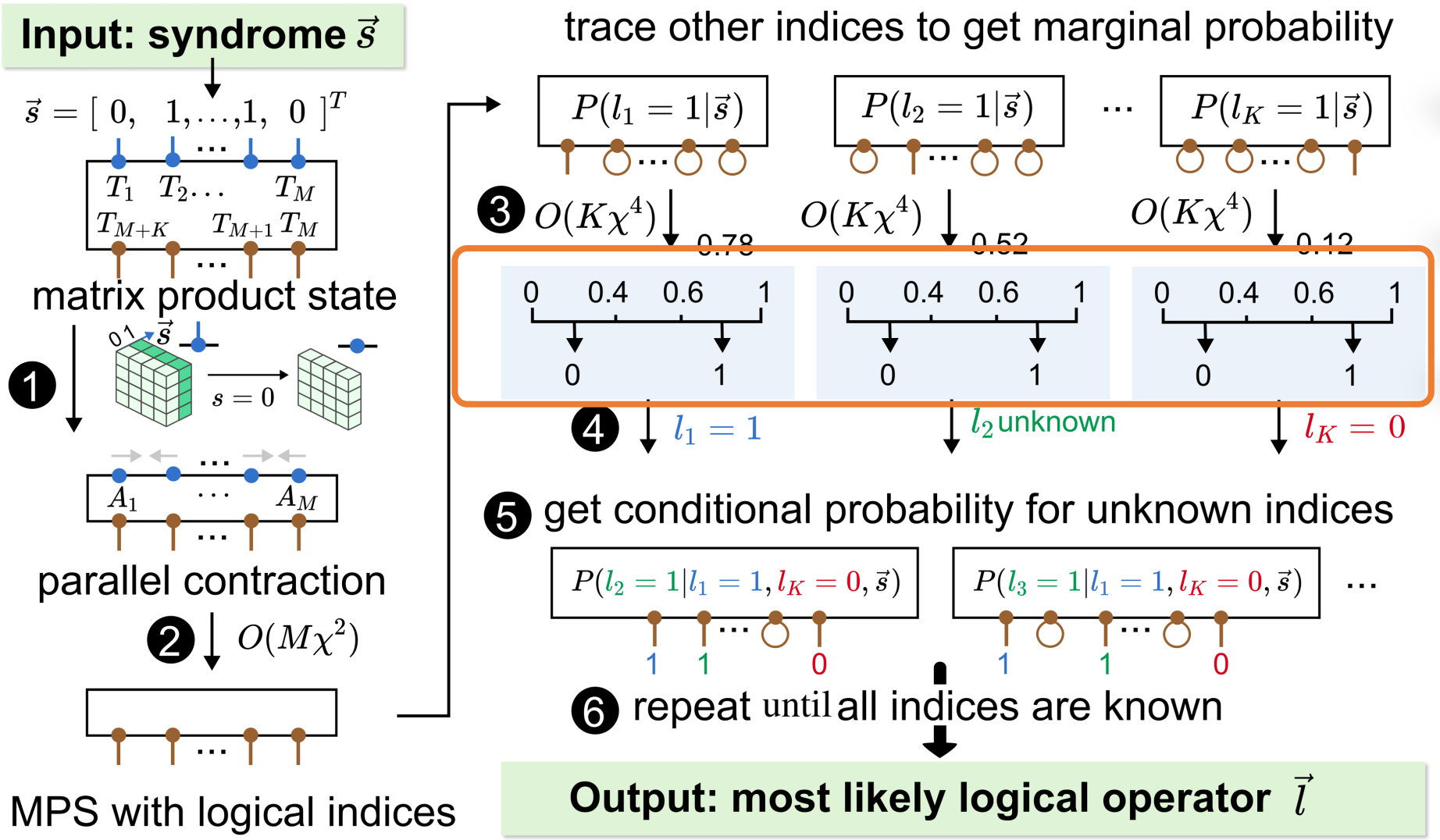


Slice by syndrome

Step 4: Online Parallel Marginal Decoding



Step 4: Online Parallel Marginal Decoding



Slice by syndrome

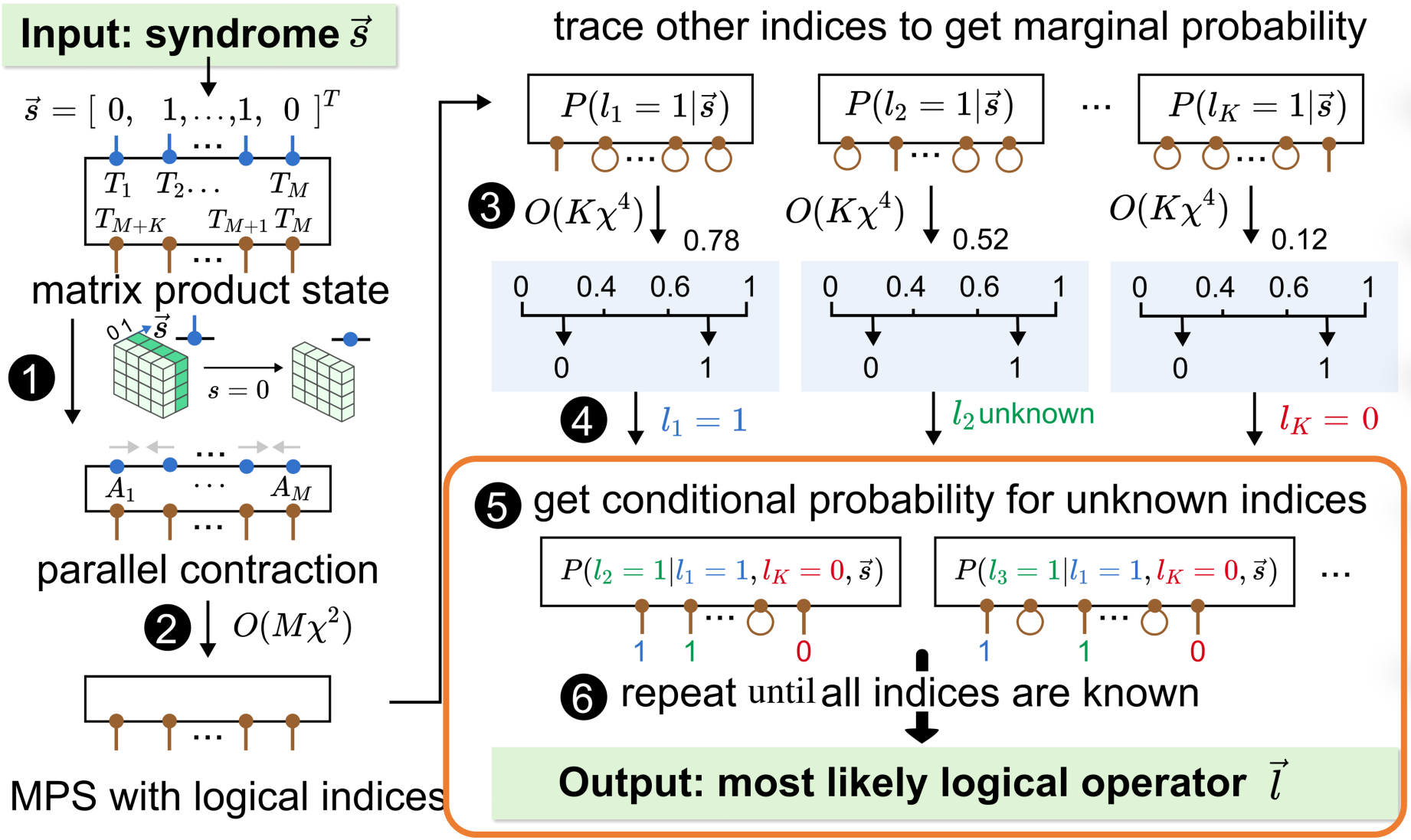
↓

Merge in parallel

↓

Marginals for all K bits

Step 4: Online Parallel Marginal Decoding



Slice by syndrome

↓

Merge in parallel

↓

Marginals for all K bits

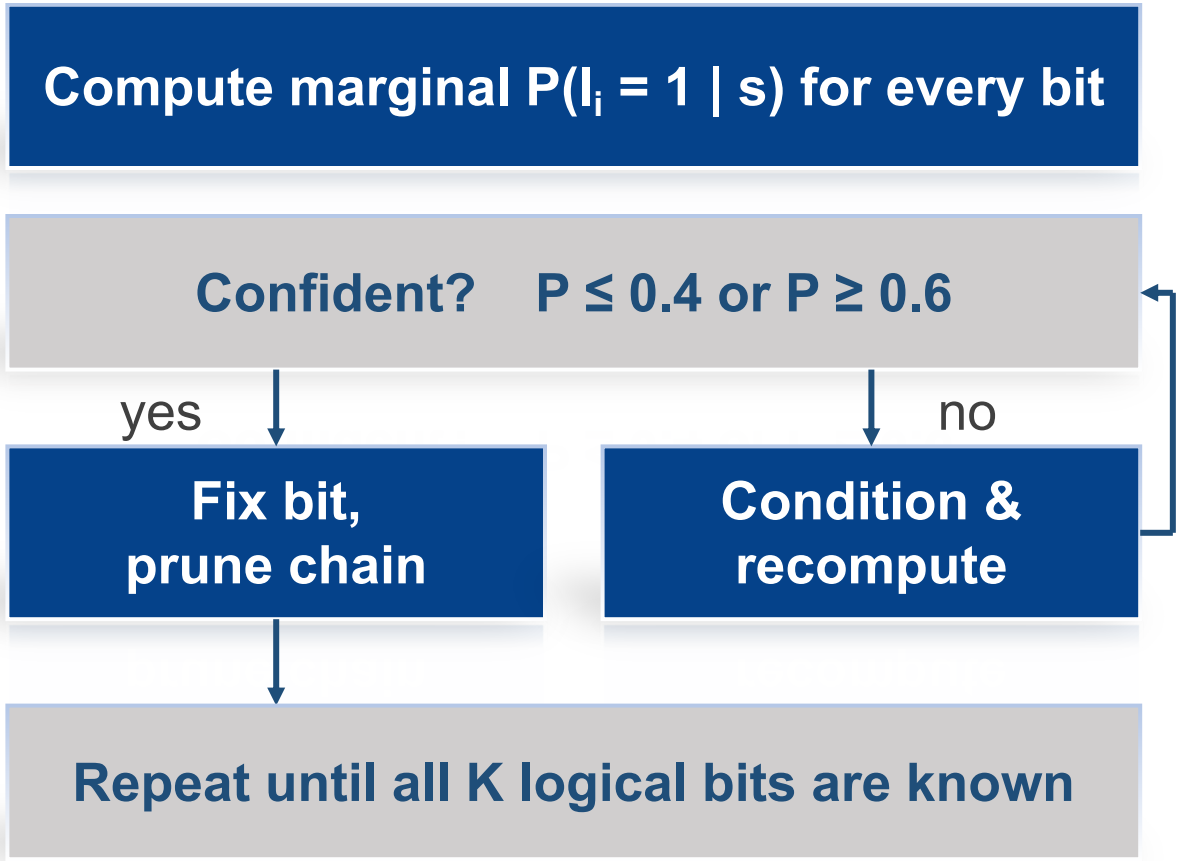
↓

Refine uncertain bits

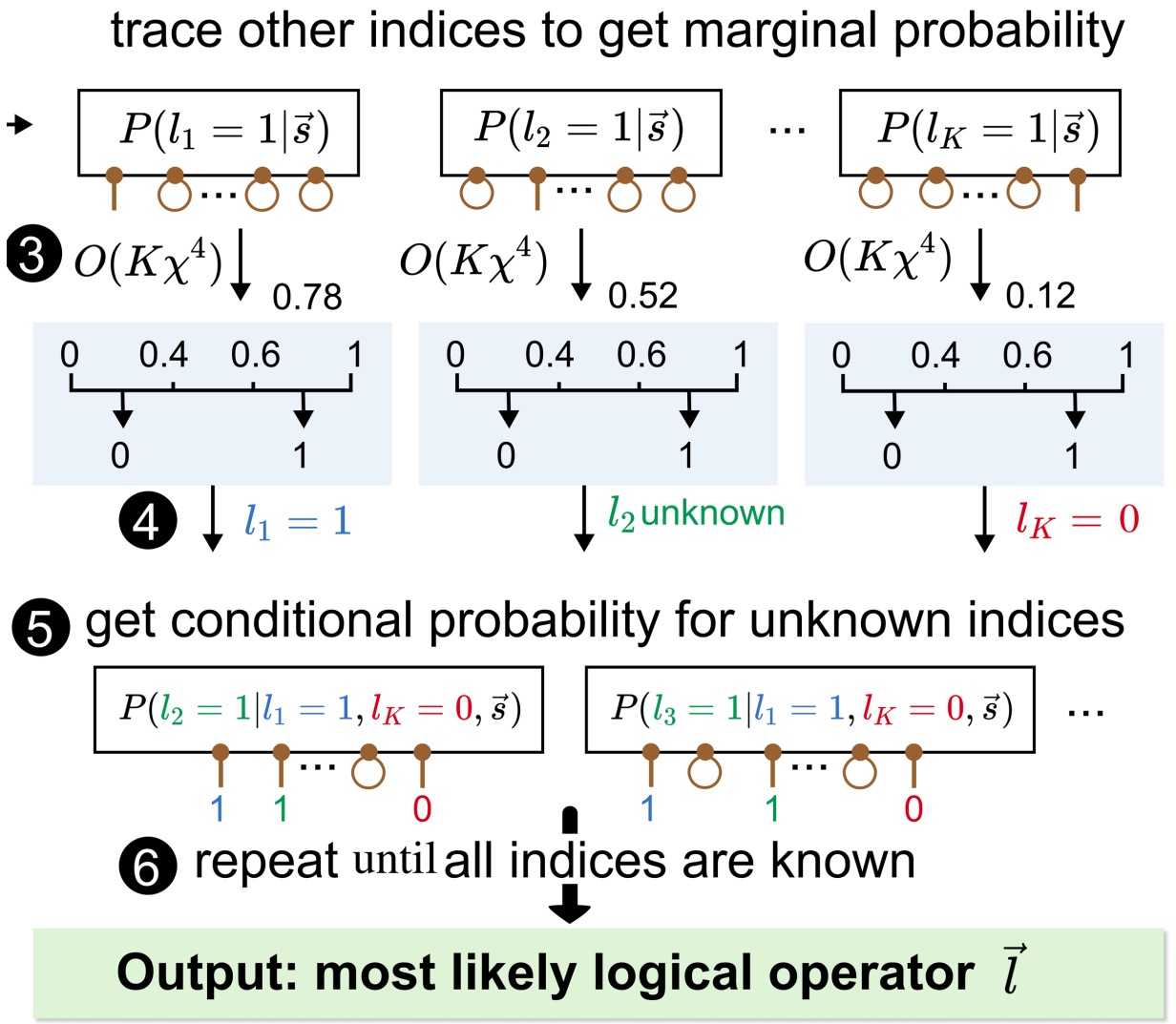
$O(M\chi^2 + \log K \cdot K\chi^4)$

no 2^K enumeration

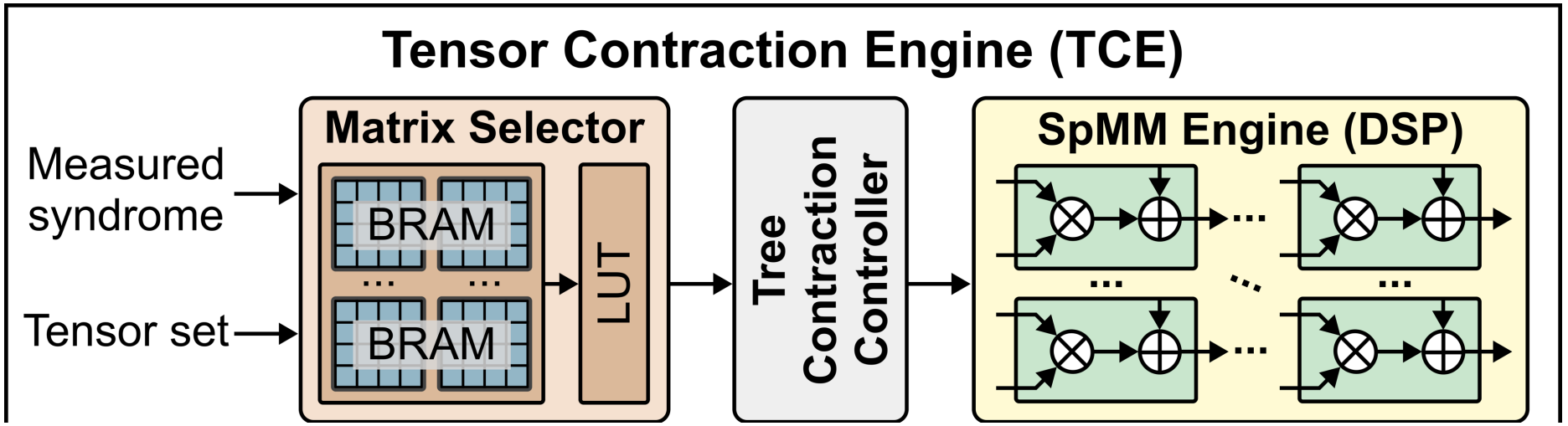
Conditional Refinement for Uncertain Logical Bits



Only unresolved bits are recomputed — correlations kept where they matter



GPU and FPGA Implementation



NVIDIA A100 GPU

block/site: syndrome → A i → **FP64 tree merge, marginals, refine**

165 – 856 µs end-to-end (H100)

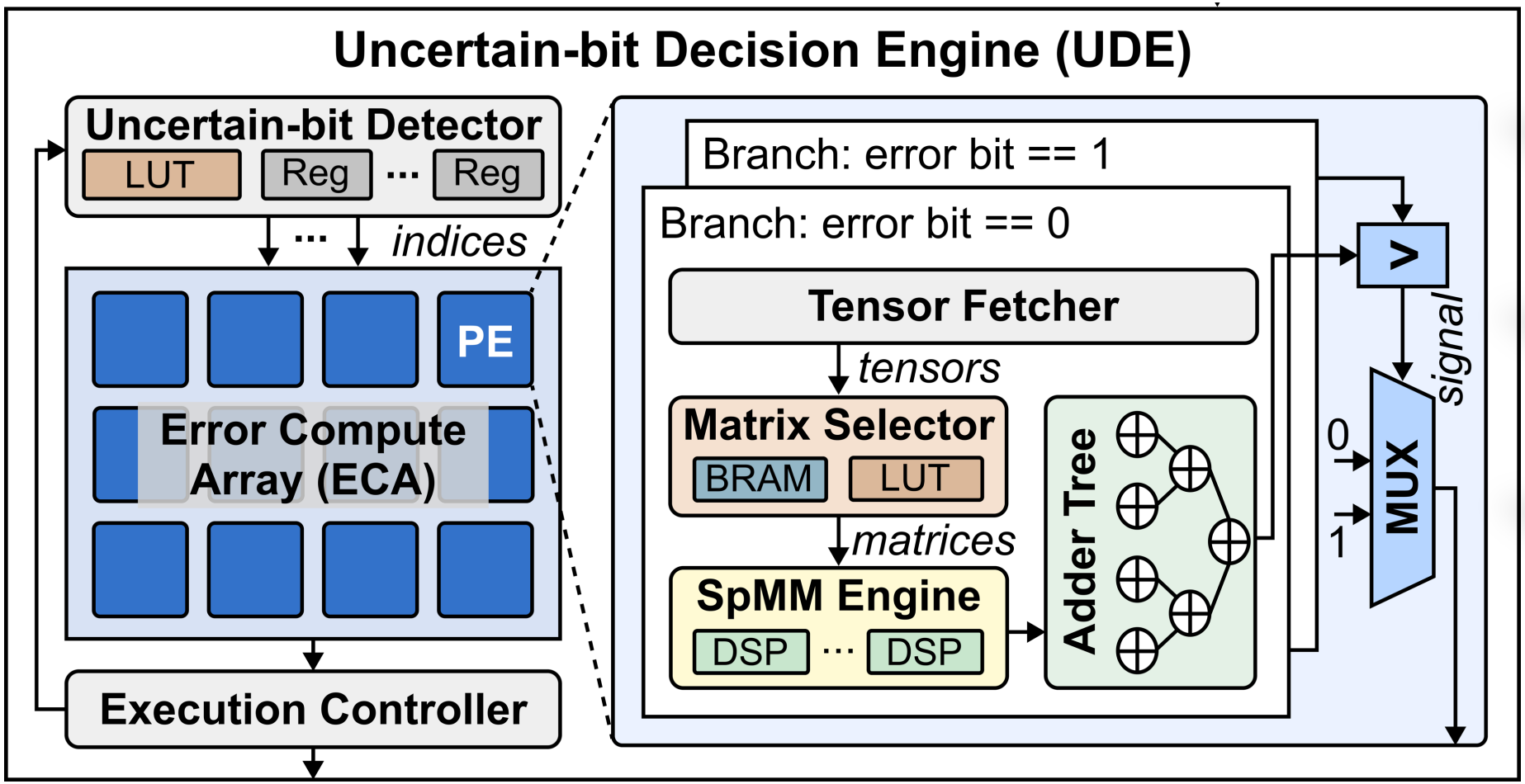
Alveo U50 FPGA

LUT+BRAM slice select → **DSP SpMM tree, PE adder tree**

0.15 – 33 µs · 250 MHz

One compressed chain deployed on GPU and FPGA

FPGA: The Uncertain-Bit Decision Engine



Uncertain-bit detector

ECA PEs: both branches

Adder tree + comparator + MUX

loop until no unknown bit

Evaluation Setup

Nine qLDPC codes

BB: [[18,4,4]] [[60,8,4]] [[72,12,6]] [[90,8,10]] [[108,8,10]] [[144,12,12]]

TB: [[25,3,4]] [[30,6,4]] [[48,4,8]]

Noise model

Stim circuit-level noise (depolarizing)
#shots: 10^4–10^7 shots
≥10 logical errors

Platforms

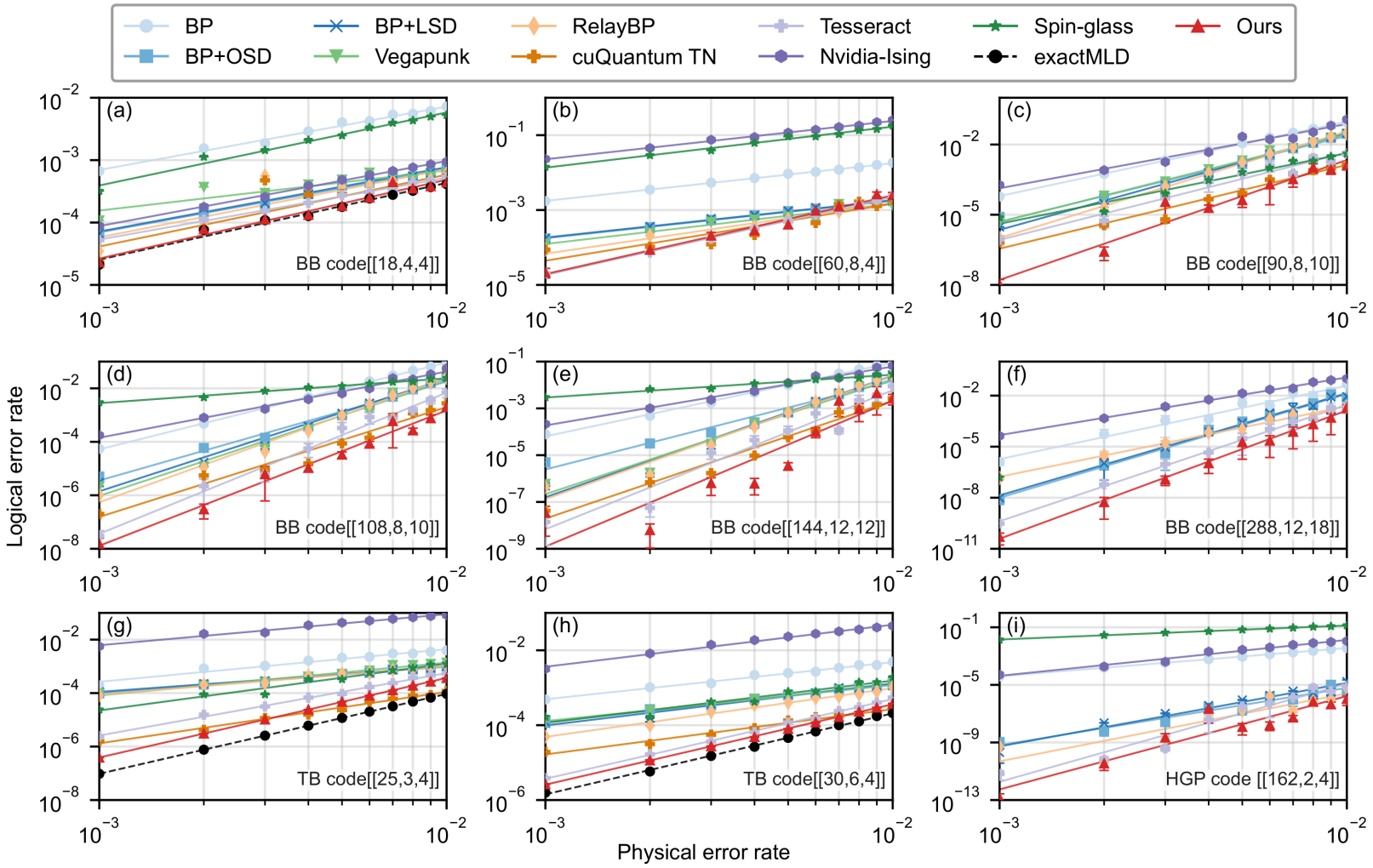
AMD EPYC 9554 CPU
A100 80GB GPU
Alveo U50 @ 250 MHz

Baselines

BP, BP+OSD, BP+LSD Vegapunk, RelayBP, CUDAQX, Spin-glass, Exact MLD

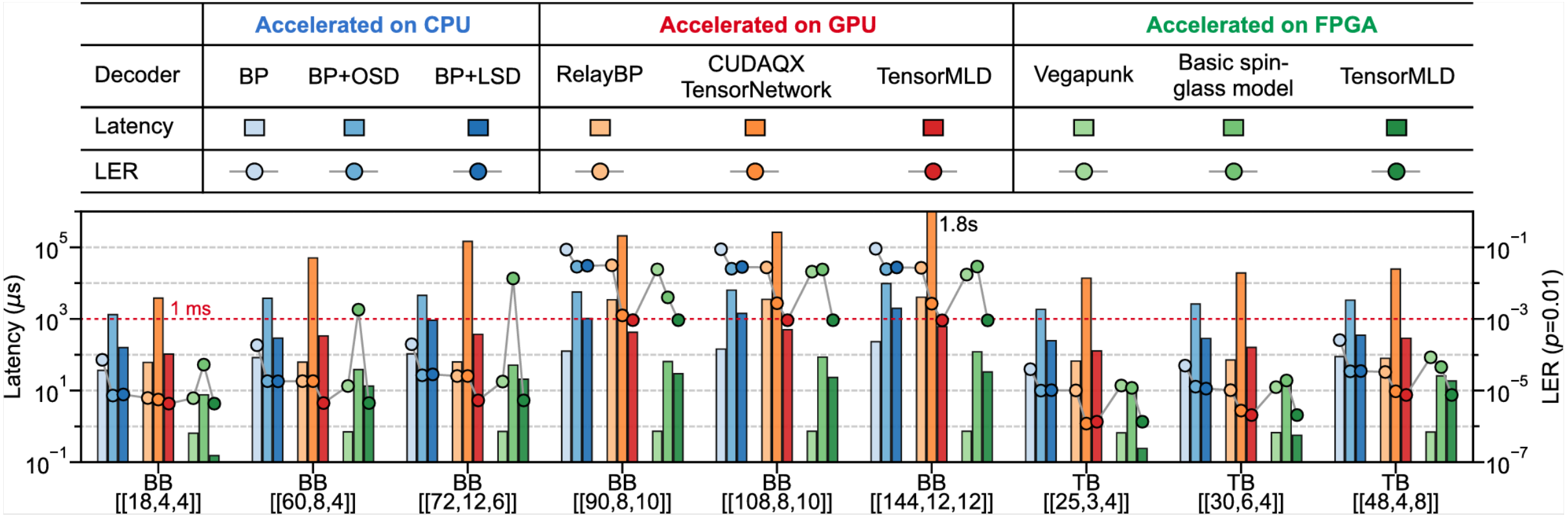
Metric	Meaning	Direction
LER	Logical failures / total trials after d syndrome rounds	Lower
Latency	The runtime of decoder, without data transfer time	Lower
FLOPs, memory	tensor computation cost	Lower

Logical Error Rate Across All Nine qLDPC Codes



Best or near-best LER on nine codes: 13.6× vs BP+OSD, 8.8× vs RelayBP (geo mean)

Latency and Tensor Compression



Decoding latency

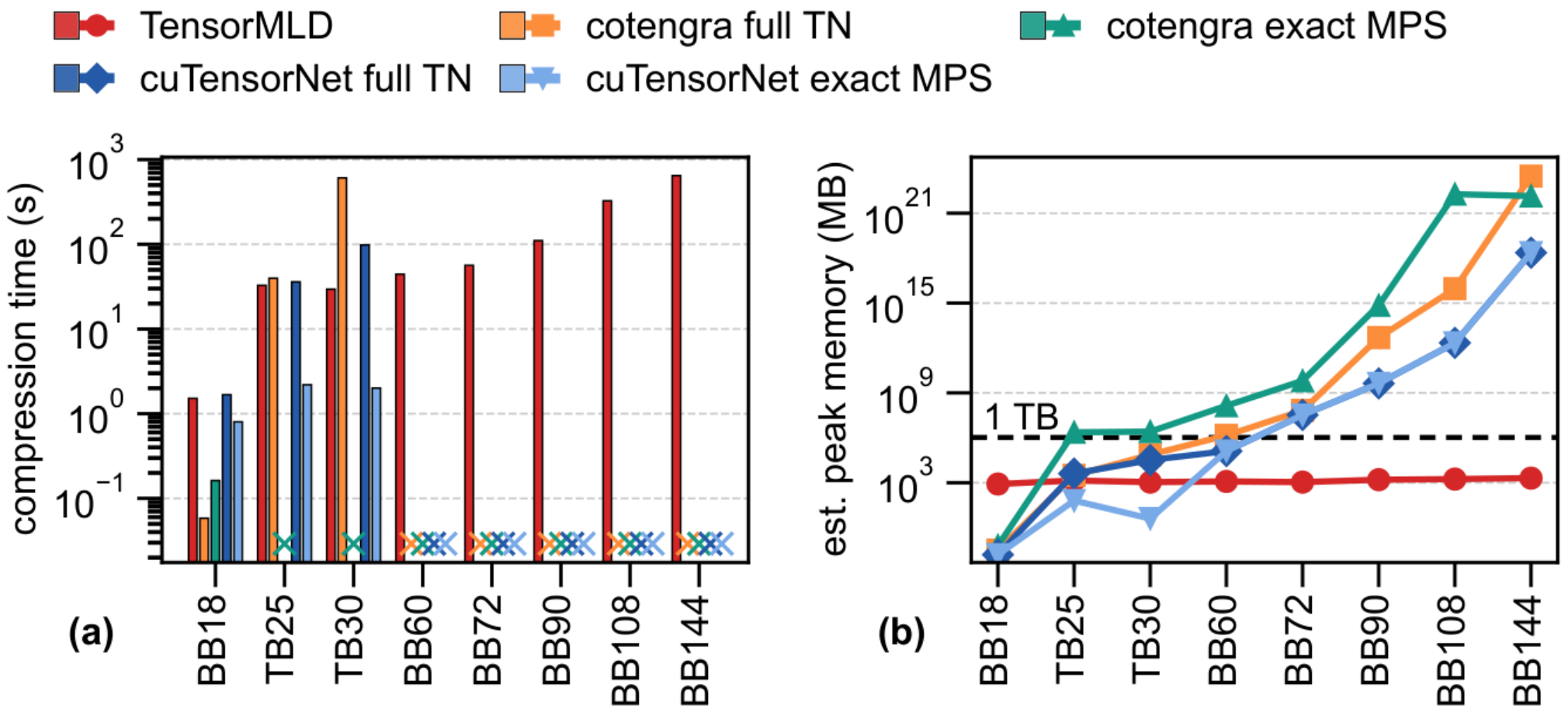
GPU (H100): 165 – 856 μs

U50 FPGA: 0.15 – 33 μs

CUDAQX TN: 3.8 ms – 146 ms

FPGA gives sub-μs on small codes, within superconducting realtime budget;
GPU decoding latency is within neutral-atom budget (10^2–10^3 μs);

Tensor Compression



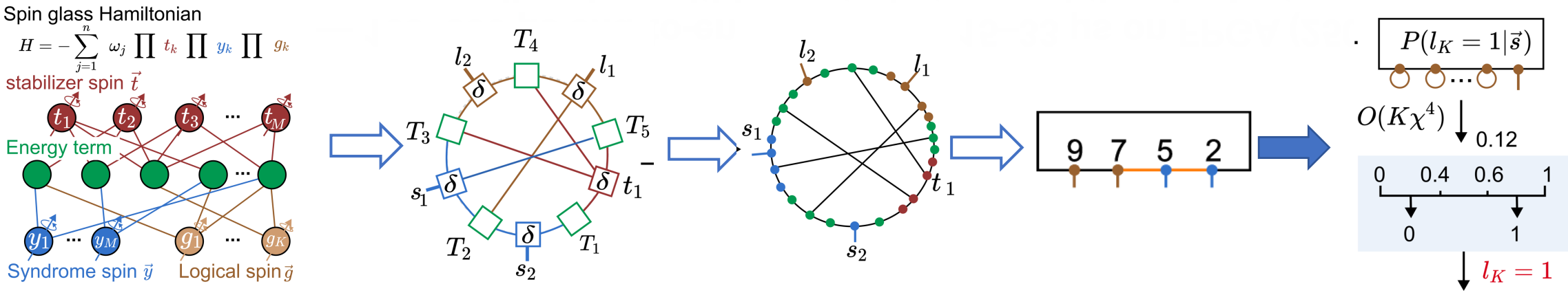
Max tensor size 1.0M → 512 (up to 2048×) · memory 3.9 GB → 8.7 KB

Summary

Accuracy — true MLD: best or near-best LER on all nine codes
13.6× lower than BP+OSD, 8.8× lower than RelayBP (geometric mean)

Cost — exact decomposition O(2ⁿ) → O(8n), offline contract–swap–merge
KFLOPs online, χ ≤ 128, no 2^K enumeration

Deployment — 165–856 μs end-to-end on GPU; 0.15–33 μs on FPGA (250 MHz)
sub-μs on small codes; within neutral-atom real-time budgets



Thanks for listening

TensorMLD: A Tensorized and Compressed qLDPC Maximum-Likelihood Decoder

Liqiang Lu*, Debin Xiang*, Kaiwen Zhou*, Boyu Jin*, Chenning Tao*, Wuwei Tian*,
Anbang Wu§, Fangxin Liu§, Jingwen Leng§, Minyi Guo§, Jianwei Yin*
*Zhejiang University; §Shanghai Jiao Tong University

MICRO 2026